

```

classDiagram
    class Cliente {
        -id int
        -nome String
        -cpf String
        -endereco Endereco
        -veiculos List<Veiculo>
    }
    class Tipousuario {
        -id int
        -nome String
        -login String
        -senha String
        -email String
        -cpf String
        -salario double
        -tipo Tipousuario
    }
    class Endereco {
        -ruaEndereco int
        -bairro Endereco
        -cidade Endereco
        -uf Endereco
        -cep Endereco
        -estado Endereco
        -cidadeEndereco Endereco
        -ruaEnderecoEndereco Endereco
        -bairroEndereco Endereco
        -cidadeEnderecoEndereco Endereco
        -ufEnderecoEndereco Endereco
        -cepEnderecoEndereco Endereco
        -estadoEnderecoEndereco Endereco
    }
    class Usuario {
        -id int
        -nome String
        -login String
        -senha String
        -email String
        -cpf String
        -salario double
        -tipo Tipousuario
    }
    class ADM_FUNCIONARIO {
        -id int
        -nome String
        -login String
        -senha String
        -email String
        -cpf String
        -salario double
        -tipo Tipousuario
    }
    class Veiculo {
        -id int
        -marca String
        -cor String
        -placa String
        -ano int
        -km int
        -done Cliente
        -servicos List<Servico>
    }
    class Servico {
        -nome String
        -preco double
    }
    class Orcamento {
        -id int
        -veiculo Veiculo
        -preco List<Preco>
        -servicos List<Servico>
        -data LocalDate
    }
    Cliente "1" -- "1" Endereco
    Cliente "1" -- "1" Veiculo
    Cliente "1" -- "1" Usuario
    Cliente "1" -- "1" ADM_FUNCIONARIO
    Cliente "1" -- "1" Veiculo
    Cliente "1" -- "1" Servico
    Cliente "1" -- "1" Orcamento
    Endereco "1" -- "1" Veiculo
    Endereco "1" -- "1" Servico
    Endereco "1" -- "1" Orcamento
    Veiculo "1" -- "1" Servico
    Veiculo "1" -- "1" Orcamento
    Servico "1" -- "1" Orcamento
    Orcamento "1" -- "1" Veiculo
    Orcamento "1" -- "1" Servico
  
```

[illegible]

```

classDiagram
    class UsuarioDAO {
        - con : Connection
        + inserir(usuario : Usuario) : void
        + deletar(usuario : Usuario) : void
        + alterar(usuario : Usuario) : void
        + listar() : List<Usuario>
    }
    class PecaDAO {
        + buscarNome(nome : String) : List<Peca>
        + buscarFabricante(fabricante : String) : List<Peca>
        + pesquisar(nome : String, fabricante : String, automovel : String) : List<Peca>
        + obterConexao() : Connection
    }
    class ServicoDAO {
        _URL : String
        _USER : String
        _PASS : String
        _con : Connection
        + (static) getConnection() : Connection
        + inserir(entites : Servico) : Servico
        + listarTodos() : List<Servico>
        + atualizar(nomeAntigo : String, novo : Servico) : void
        + deletar(nome : String) : void
        + buscarPorNome(nome : String) : List<Servico>
    }
    class VeiculoDAO {
        - con : Connection
        + inserir(veiculo : Veiculo) : void
        + deletar(veiculo : Veiculo) : void
        + alterar(veiculo : Veiculo) : void
        + listar() : List<Veiculo>
        + buscarId(id : int) : ArrayList<Veiculo>
        + buscarPorPlaca(placa : String) : ArrayList<Veiculo>
        + buscarPorMarca(marca : String) : ArrayList<Veiculo>
        + buscarPorCliente(idCliente : int) : ArrayList<Veiculo>
        + buscarPorCor(cor : String) : ArrayList<Veiculo>
        + buscarPorAno(ano : int) : ArrayList<Veiculo>
        + buscarPorKm(kmMin : int, kmMax : int) : ArrayList<Veiculo>
        + deletarPorPlaca(placa : String) : void
        + existePlaca(placa : String) : boolean
        + criarVeiculo(rs : ResultSet) : Veiculo
    }
    class ClienteDAO {
        + inserir(cliente : Cliente) : void
        + deletar(cliente : Cliente) : void
        + alterar(cliente : Cliente) : void
        + listar() : List<Cliente>
        + buscarId(id : int) : ArrayList<Cliente>
        + buscarCpf(cpf : String) : String
        + mapearCliente(rs : ResultSet) : Cliente
    }
    class EnderecoDAO {
        - con : Connection
        + inserir(endereco : Endereco) : void
        + deletar(endereco : Endereco) : void
        + alterar(endereco : Endereco) : void
        + listar() : List<Endereco>
        + buscarId(id : int) : ArrayList<Endereco>
        + buscarPorRua(rua : String) : ArrayList<Endereco>
        + buscarPorBairro(bairro : String) : ArrayList<Endereco>
    }
    class OrcamentoDAO {
        + inserir(orcamento : Orcamento) : void
        + deletar(orcamento : Orcamento) : void
        + alterar(orcamento : Orcamento) : void
        + listar() : List<Orcamento>
        + inserir(orcamento : Orcamento) : void
        + deletar(orcamento : Orcamento) : void
        + alterar(orcamento : Orcamento) : void
        + listar() : List<Orcamento>
        + salvarPecasDoOrcamento(idOrcamento : int, pecas : List<Peca>) : void
        + salvarServicosDoOrcamento(idOrcamento : int, servicos : List<Servico>) : void
        + buscarServicosDoOrcamento(idOrcamento : int) : List<Servico>
        + buscarPecasDoOrcamento(idOrcamento : int) : List<Peca>
    }
    class OrdemDeServicoDAO {
        - orcamentoDAO : OrcamentoDAO
        + inserir(os : OrdemServico) : OrdemServico
        + atualizarStatus(os : OrdemServico) : void
        + deletar(idOrcamento : int) : void
        + listarTodos() : List<OrdemServico>
        + obterConexao() : Connection
    }
    UsuarioDAO ..> BaseDAO
    PecaDAO ..> BaseDAO
    ServicoDAO ..> BaseDAO
    VeiculoDAO ..> BaseDAO
    ClienteDAO ..> BaseDAO
    EnderecoDAO ..> BaseDAO
    OrcamentoDAO ..> BaseDAO
    OrdemDeServicoDAO ..> BaseDAO
    BaseDAO {
        + getConnection() : Connection
        + inserir(E) : void
        + deletar(E) : void
        + alterar(E) : void
        + listar() : List<E>
    }

```

```

classDiagram
    class ClienteService {
        -clienteDAO : ClienteDAO
        -enderecoDAO : EnderecoDAO
        +cadastrarCliente(cliente : Cliente) : void
        +buscarPorCpf(cpf : String) : Cliente
        +listarClientes() : List<Cliente>
        +alterarCliente(cpfAntigo : String, novoNome : String, novoCpf : String, novoEndereco : Endereco) : void
        +alterarNome(cpf : String, novoNome : String) : void
        +alterarCpf(cpfAtual : String, novoCpf : String) : void
        +removerCliente(cpf : String) : boolean
        +alterarEndereco(cpf : String, novoEndereco : Endereco) : void
        +alterarRua(cpf : String, novaRua : String) : void
        +alterarBairro(cpf : String, novoBairro : String) : void
        +alterarNumero(cpf : String, novoNumero : int) : void
    }

    class OrcamentoService {
        -orcamentos : List<Orcamento>
        +adicionar(o : Orcamento) : void
        +listarTodos() : List<Orcamento>
        +editarPorPlaca(placa : String, novoOrcamento : Orcamento) : boolean
        +excluirPorPlaca(placa : String) : boolean
        +buscarPorVeiculo(placa : String) : List<Orcamento>
        +buscarPorCliente(nomeCliente : String) : List<Orcamento>
        +buscarPorPeriodo(inicio : LocalDate, fim : LocalDate) : List<Orcamento>
    }

    class OrdemServiceService {
        -osDAO : OrdemDeServicoDAO
        +adicionar(os : OrdemServico) : void
        +editarOrdem(os : OrdemServico) : void
        +excluirOrdem(idOrcamento : int) : void
        +listarTodasOrdens() : List<OrdemServico>
    }

    class PecaService {
        -pecaDAO : PecaDAO
        +adicionar(peca : Peca) : void
        +alterar(peca : Peca) : void
        +deletar(peca : Peca) : void
        +listarTodos() : List<Peca>
        +buscarPorNome(nome : String) : List<Peca>
        +buscarPorFabricante(fabricante : String) : List<Peca>
        +pesquisar(nome : String, fabricante : String, automovel : String) : List<Peca>
    }

    class VeiculoService {
        -veiculoDAO : VeiculoDAO
        +cadastarVeiculo(veiculo : Veiculo) : void
        +buscarPorPlaca(placa : String) : Veiculo
        +buscarPorDono(cpf : String) : List<Veiculo>
        +listarVeiculos() : List<Veiculo>
        +alterarVeiculo(placaAntiga : String, novaMarca : String, novaCor : String, novaPlaca : String, novoAno : int, novoKm : int) : void
        +alterarMarca(placa : String, novaMarca : String) : void
        +alterarCor(placa : String, novaCor : String) : void
        +alterarPlaca(placa : String, novaPlaca : String) : void
        +alterarAno(placa : String, novoAno : int) : void
        +alterarKm(placa : String, novoKm : int) : void
        +removeVeiculo(placa : String) : boolean
    }

    class RelatorioFactory {
        +obterRelatorio(tipo)
    }

    class RelatorioFaturamentoService {
        -osService : OrdemServicoService
        +gerar() : RelatorioFaturamento
        +calcularFaturamentoTotal(ordens : List<OrdemServico>) : double
        +calcularTotalPecas(ordens : List<OrdemServico>) : double
        +calcularTotalServicos(ordens : List<OrdemServico>) : double
        +contarClientesUnicos(ordens : List<OrdemServico>) : long
    }

    class RelatorioService {
        -osService : OrdemServicoService
        +obterOrdensFaturadas() : List<OrdemServico>
        +calcularFaturamentoTotal(ordens : List<OrdemServico>) : double
        +calcularTotalPecas(ordens : List<OrdemServico>) : double
        +calcularTotalServicos(ordens : List<OrdemServico>) : double
        +contarClientesUnicos(ordens : List<OrdemServico>) : long
    }

    class Relatorio {
        <<interface>>
    }

    class RelatorioFaturamento {
    }

    RelatorioFactory --> Relatorio : cria
    RelatorioFaturamento ..> Relatorio : 
    
```

ClienteService

- clienteDAO : ClienteDAO
- enderecoDAO : EnderecoDAO
- + cadastrarCliente(cliente : Cliente) : void
- + buscarPorCpf(cpf : String) : Cliente
- + listarClientes() : List<Cliente>
- + alterarCliente(cpfAntigo : String, novoNome : String, novoCpf : String, novoEndereco : Endereco) : void
- + alterarNome(cpf : String, novoNome : String) : void
- + alterarCpf(cpfAtual : String, novoCpf : String) : void
- + removerCliente(cpf : String) : boolean
- + alterarEndereco(cpf : String, novoEndereco : Endereco) : void
- + alterarRua(cpf : String, novaRua : String) : void
- + alterarBairro(cpf : String, novoBairro : String) : void
- + alterarNumero(cpf : String, novoNumero : int) : void

OrcamentoService

- orcamentos : List<Orcamento>
- + adicionar(o : Orcamento) : void
- + listarTodos() : List<Orcamento>
- + editarPorPlaca(placa : String, novoOrcamento : Orcamento) : boolean
- + excluirPorPlaca(placa : String) : boolean
- + buscarPorVeiculo(placa : String) : List<Orcamento>
- + buscarPorCliente(nomeCliente : String) : List<Orcamento>
- + buscarPorPeriodo(inicio : LocalDate, fim : LocalDate) : List<Orcamento>

OrdemServiceService

- osDAO : OrdemDeServicoDAO
- + adicionar(os : OrdemServico) : void
- + editarOrdem(os : OrdemServico) : void
- + excluirOrdem(idOrcamento : int) : void
- + listarTodasOrdens() : List<OrdemServico>

PecaService

- pecaDAO : PecaDAO
- + adicionar(peca : Peca) : void
- + alterar(peca : Peca) : void
- + deletar(peca : Peca) : void
- + listarTodos() : List<Peca>
- + buscarPorNome(nome : String) : List<Peca>
- + buscarPorFabricante(fabricante : String) : List<Peca>
- + pesquisar(nome : String, fabricante : String, automovel : String) : List<Peca>

VeiculoService

- veiculoDAO : VeiculoDAO
- + cadastarVeiculo(veiculo : Veiculo) : void
- + buscarPorPlaca(placa : String) : Veiculo
- + buscarPorDono(cpf : String) : List<Veiculo>
- + listarVeiculos() : List<Veiculo>
- + alterarVeiculo(placaAntiga : String, novaMarca : String, novaCor : String, novaPlaca : String, novoAno : int, novoKm : int) : void
- + alterarMarca(placa : String, novaMarca : String) : void
- + alterarCor(placa : String, novaCor : String) : void
- + alterarPlaca(placa : String, novaPlaca : String) : void
- + alterarAno(placa : String, novoAno : int) : void
- + alterarKm(placa : String, novoKm : int) : void
- + removeVeiculo(placa : String) : boolean

RelatorioFactory

- + obterRelatorio(tipo)

RelatorioFaturamentoService

- osService : OrdemServicoService
- + gerar() : RelatorioFaturamento
- + calcularFaturamentoTotal(ordens : List<OrdemServico>) : double
- + calcularTotalPecas(ordens : List<OrdemServico>) : double
- + calcularTotalServicos(ordens : List<OrdemServico>) : double
- + contarClientesUnicos(ordens : List<OrdemServico>) : long

RelatorioService

- osService : OrdemServicoService
- + obterOrdensFaturadas() : List<OrdemServico>
- + calcularFaturamentoTotal(ordens : List<OrdemServico>) : double
- + calcularTotalPecas(ordens : List<OrdemServico>) : double
- + calcularTotalServicos(ordens : List<OrdemServico>) : double
- + contarClientesUnicos(ordens : List<OrdemServico>) : long

<<interface>> Relatorio

RelatorioFaturamento

ServicoService

- servicoDAO : ServicoDAO
- + adicionar(servico : Servico) : void
- + listarTodos() : List<Servico>
- + alterar(nomeAntigo : String, novoServico : Servico) : void
- + deletar(nome : String) : void
- + buscarPorNome(nome : String) : List<Servico>

UsuarioService

- usuarioDAO : UsuarioDAO
- + registrarUsuario(quem : Usuario, novo : Usuario) : void
- + buscarPorId(id : int) : Usuario
- + buscarPorLogin(login : String) : Usuario
- + listarTodos() : List<Usuario>
- + atualizarUsuario(quem : Usuario, id : int, dados : Usuario) : void
- + excluirUsuario(quem : Usuario, id : int) : void
- + fazerLogin(login : String, senha : String) : Usuario
- + verificarSeAdmin(usuario : Usuario,acao : String) : void

Annotations:

- List<Orcamento>
List é o Aggregate
- O Iterator está implícito dentro dela